UNIVERSITY
OF WOLLONGONG
IN DUBAI

Project Cover Sheet

Student name	Student number	Student submitting work (x)
1. Zainab Khan	8442861	x
2. Neha Shababudeen	8388635	

Subject number and name:	CSCI291 - Programming for Enginners
Subject coordinator:	Dr. Abdsamad Benkrid
Title of Assignment:	Webots Robot Application: Maze Solving
Date and time due:	24/11/2024 @ 23:00
Tutor Name:	Mr. Kiyan Afsari
Total number of pages:	12

Student declaration and acknowledgment (must be read by all students)

By submitting this assignment online, the submitting student declares on behalf of the team that:

1. All team members have read the subject outline for this subject, and this assessment item meets the requirements of the subject detailed therein.
2. This assessment is entirely our work, except where we have included fully documented references to the work of others. The material contained in this assessment item has yet to be submitted for assessment.
3. Acknowledgment of source information is in accordance with the guidelines or referencing style specified in the subject outline.
4. All team members are aware of the late submission policy and penalty.
5. The submitting student undertakes to communicate all feedback with the other team members.

<u>Webots Project Report:</u>

Maze-Solving Robot

CSCI291 - Programming for Engineers

School of Engineering
University of Wollongong in Dubai

Zainab Khan - 8442861
Neha Shababudeen - 8388635

Sunday, November 24th, 2024

Contents:

1. ***Introduction & Aim……4***
2. ***Adaptation of Algorithm……4***

   2.1. Right-Hand Wall Follower……4

   2.2. Smooth Movement Control……..……5

3. ***Implementation of Functions……..…6***

   3.1. Robot Set Up…….6

   3.2. Sensor Readings & Dead-End Detection…….7

   3.3. Finding & Navigating to Target Position……9

   3.4 Movement Control & Position Tracking…….10

   3.5 Data Management & Processing……..10

4. ***Conclusion……11***
5. ***References……11***
6. ***Appendix A……..11***

1. Introduction & Aim

The goal of this project is to design and program an autonomous maze-solving robot using Webots. The focus is to create an efficient robot capable of navigating complex mazes and record the varying light intensities throughout to identify the point with the maximum light intensity.

This is achieved by integrating several proximity and light sensors, and a GPS, along with the use of the right-hand wall following algorithm.

The implementation is carried out in C, and not hardcoded for any specific maze configuration.

2. Adaptation of Algorithm

2.1. Right-Hand Wall Follower

```
//right-hand wall-following behavior
double prox_values[8];
for (int i = 0; i < 8; ++i) {
  prox_values[i] = wb_distance_sensor_get_value(prox_sensors[i]);  //read all proximity sensors
}

bool front_wall = prox_values[7] > WALL_THRESHOLD;  //check if wall is detected in front
bool left_wall = prox_values[1] > WALL_THRESHOLD;  //check if wall is detected on left

//control motor speeds based on wall detection
if (front_wall) {
  right_speed = MAX_SPEED;  //turn left if front wall detected
  left_speed = -MAX_SPEED;
} else if (left_wall) {
  right_speed = MAX_SPEED;  //follow  left wall if left wall detected
  left_speed = MAX_SPEED;
} else {
  right_speed = MAX_SPEED / 8;  //go forward slowly when no walls are detected
  left_speed = MAX_SPEED;
}
```

RIGHT-HAND WALL-FOLLOWING ALGORITHM

The **Right-Hand Wall-Follower Algorithm** is a maze exploration strategy where the robot follows the wall on its right side. The algorithm utilizes data from eight proximity sensors placed around the robot to identify walls and obstacles. The robot's behaviour depends on the presence of a wall in front(monitored by sensor ps7), or absence of wall on its right(monitored by sensor ps1).
Here’s how it works:

1. **Move Forward When Possible**:
   If the robot detects a wall on its right but no obstacle directly in front, it simply drives forward with both motors running at full speed, staying parallel to the right wall.
2. **Handle Obstacles in Front (Corners)**:
   If the robot detects a wall directly in front of it(prox_values[7] > WALL_THRESHOLD), it means it has reached a corner. To continue following the right-hand wall, it turns **left** to avoid the obstacle by adjusting the motor speeds accordingly (right_speed = MAX_SPEED, left_speed = -MAX_SPEED)and re-align with the right-hand wall.
3. **Take Right Turns**:
   If the right-hand wall disappears (indicating an upcoming right turn), the robot turns **right** to follow the new right wall.
4. **Handle U-Turns**:
   The same logic for right turns also applies to U-turns. If there is no wall on the right and no wall in front, the robot takes a sharp **right turn** to find and follow the right wall again.
5. **Adjust for No Wall on the Right and an Obstacle Ahead**:
   If there is no wall on the right but the robot detects an obstacle directly in front, it turns **left on the spot** slowly by making the left motor run at top speed and decreasing the right motor's speed to MAX_SPEED/8 until it aligns with a new right-hand wall.

This simple set of rules allows the robot to explore mazes while consistently following the right-hand wall.

2.2. Smooth Movement Control

```
#define SMOOTHING_FACTOR 0.07   //factor for smoothing transitions in motor speed
```

The smooth movement control system is implemented through the smooth_speed( ) function, which utilizes a SMOOTHING_FACTOR constant set at 0.07 for gradual speed transitions between motors.

```
double smooth_speed(double current_speed, double target_speed) {
  return current_speed + SMOOTHING_FACTOR * (target_speed - current_speed);
}
```

This system smooths out abrupt movements in the robot, leading to decreased mechanical strain and enhanced navigation stability. The function computes the updated velocity by blending the current velocity and the desired velocity through the equation: current_speed + SMOOTHING_FACTOR * (target_speed - current_speed).

```
//smoothly adjust motor speeds
right_speed = smooth_speed(right_speed, MAX_SPEED);
left_speed = smooth_speed(left_speed, MAX_SPEED);

wb_motor_set_velocity(right_motor, right_speed);  //set right motor speed
wb_motor_set_velocity(left_motor, left_speed);  //set left motor speed
```

Precise control over the robot's turning radius and forward motion is achieved by separately applying smooth movement control to both motors. This application is especially important when transitioning between various movement states, like switching from moving forward to turning or modifying the robot's trajectory while tracking a wall. The system ensures continuous contact with the walls during turns and avoids wheel slipping, leading to more dependable navigation in the maze.

3. Implementation of Functions

3.1. Robot Set Up

```
//main loop
int main(int argc, char **argv) {
    //initialize robot
  wb_robot_init();

  //set up motor devices for controlling robot's wheels
  WbDeviceTag right_motor = wb_robot_get_device("right wheel motor");
  WbDeviceTag left_motor = wb_robot_get_device("left wheel motor");
  wb_motor_set_position(right_motor, INFINITY);  //set motors to continuous rotation
  wb_motor_set_position(left_motor, INFINITY);  //set motors to continuous rotation
  wb_motor_set_velocity(right_motor, 0.0);  //set initial velocity to 0 for both motors
  wb_motor_set_velocity(left_motor, 0.0);   //set initial velocity to 0 for both motors
```

The robot is initialized systematically by starting with wb_robot_init() to set up the basic functions of the robot. The initial vital task is to set up the motors by configuring both the left and right wheel motors using wb_robot_get_device() and initializing them with wb_motor_set_position(INFINITY) for constant rotation.

```
//set up proximity sensors to detect obstacles around the robot
WbDeviceTag prox_sensors[8];
char prox_sensor_name[50];
for (int ind = 0; ind < 8; ++ind) {  //loop over each proximity sensor
  sprintf(prox_sensor_name, "ps%d", ind);  //construct sensor name dynamically ps0 to ps7
  prox_sensors[ind] = wb_robot_get_device(prox_sensor_name);  //get sensor device
  wb_distance_sensor_enable(prox_sensors[ind], TIME_STEP);  //enable sensor with given time step
}
```

The second operation creates a set of eight proximity sensors through a loop which sets up each sensor with a distinct label ("ps0" to "ps7") and activates them with the designated time interval.

```
//set up GPS device for tracking robot's position
WbDeviceTag gps = wb_robot_get_device("gps");
wb_gps_enable(gps, TIME_STEP);  //enable GPS with the given time step
```

```
//set up light sensors to measure light intensity in the environment
WbDeviceTag light_sensors[NUM_SENSORS];
char sensor_name[5];
for (int i = 0; i < NUM_SENSORS; i++) {  //loop over each light sensor
  sprintf(sensor_name, "ls%d", i);  //construct sensor name
  light_sensors[i] = wb_robot_get_device(sensor_name);  //get light sensor device
  wb_light_sensor_enable(light_sensors[i], TIME_STEP);  //enable light sensor with given time step
}
```

The GPS and light sensors are set up using comparable device acquisition and enabling functions, with the light sensors playing a vital role in detecting changes in light intensity across the maze.

```
//initialize motor speeds
double right_speed = MAX_SPEED;  //set right motor speed to maximum speed
double left_speed = MAX_SPEED;   //set left motor speed to maximum speed
```

3.2. Sensor Readings & Dead-End Detection

```
//struct to store coordinates of detected dead ends
typedef struct {
  double x; //x-coordinate of dead end
  double y; //y-coordinate of dead end
  double z; //z-coordinate of dead end
} DeadEnd;

//array to store detected dead ends //size 25
DeadEnd detected_dead_ends[25];
int num_dead_ends = 0;  //variable to track number of detected dead ends
```

The system keeps track of detected dead ends in the array called detected_dead_ends and logs positions with their corresponding light intensities in the recorded_positions array to fully map the maze environment.

```
//function to check if current sensor readings indicate dead end
bool is_dead_end(double prox_values[]) {
  bool front_wall = prox_values[7] > WALL_THRESHOLD;  //check if there is a wall in front of the robot
  bool left_wall = prox_values[1] > DEAD_END_THRESHOLD;  //check if there is a wall on the left
  bool right_wall = prox_values[5] > DEAD_END_THRESHOLD;  //check if there is a wall on the right

  //return true if walls are detected on three sides
  if (front_wall && left_wall && right_wall) {
    return true;
  }

  //if walls are on left and right but not in front, check if it is a narrow passage
  if ((left_wall && right_wall) && !front_wall) {
    double front_distance = prox_values[7];
    if (front_distance > WALL_THRESHOLD && front_distance < DEAD_END_THRESHOLD) {
      return true;
    }
  }

  return false;  //return false if no dead end detected
}
```

The system uses multiple specific functions for managing sensors and detecting dead ends. The main purpose of is_dead_end() is to analyze sensor readings compared to set thresholds to determine if a dead end situation has occurred, taking into account regular three-wall setups and narrow passages.

```
//function to check if a current position is a previously recorded dead end
bool is_duplicate_dead_end(const double *position) {
  for (int i = 0; i < num_dead_ends; i++) {
    double dist = distance_between_points(position, (double[]){detected_dead_ends[i].x,
    detected_dead_ends[i].y, detected_dead_ends[i].z});
    if (dist < DEAD_END_PROXIMITY_THRESHOLD) {  /*if current position is too close to a
    previously recorded dead end*/
      return true;
    }
  }
  return false;  //return false if no duplicate dead end detected
}
```

The is_duplicate_dead_end() function uses the distance_between_points() utility function to avoid repetitive dead end detections by comparing current positions with previously detected dead ends.

```
At Position: (-1.42, 0.33, 0.00), Average Light Intensity: 3046.12
At Position: (-1.42, 0.33, 0.00), Average Light Intensity: 3043.18
At Position: (-1.43, 0.33, 0.00), Average Light Intensity: 3013.42
At Position: (-1.43, 0.34, 0.00), Average Light Intensity: 3000.92
At Position: (-1.43, 0.34, 0.00), Average Light Intensity: 2995.38
At Position: (-1.43, 0.34, 0.00), Average Light Intensity: 3046.01
At Position: (-1.44, 0.34, 0.00), Average Light Intensity: 2986.65
At Position: (-1.44, 0.34, 0.00), Average Light Intensity: 2955.91 [DEAD-END]
At Position: (-1.44, 0.33, 0.00), Average Light Intensity: 3117.37
At Position: (-1.44, 0.33, 0.00), Average Light Intensity: 3115.40
At Position: (-1.45, 0.33, 0.00), Average Light Intensity: 3114.31
```

The robot also has 8 light sensors (defined by NUM_SENSORS).Each sensor reading is obtained using, wb_light_sensor_get_value(). The code then sums up all the readings and gets the average value.

```
double average_light_value = light_sum / NUM_SENSORS;
```

3.3. Finding & Navigating to Target Position

```
//struct to store position and light intensity at coordinates
typedef struct {
  double x; // x-coordinate of the position
  double y; // y-coordinate of the position
  double z; // z-coordinate of the position
  double light_intensity; //light intensity measured at position
} PositionLightIntensity;

//array to store recorded positions and light intensities
PositionLightIntensity recorded_positions[MAX_POSITIONS];
int num_recorded_positions = 0;  //variable to track number of recorded positions
```

The target position navigation system operates using a complete set of functions that work together. At first, the system saves the positions and light intensities during the maze exploration stage, storing them in the recorded_positions array.

```
//function to check if robot is back at starting position
bool is_back_at_start() {
  const double *current_position = wb_gps_get_values(wb_robot_get_device("gps"));
  //get current position from GPS
  double dist = distance_between_points(current_position, initial_position);
  //calculate distance from initial position
  return dist < 0.5;  //return true if distance is less than 0.5 meters
}
```

```
//function to calculate distance between points
double distance_between_points(const double *pos1, const double *pos2) {
  return sqrt(pow(pos2[0] - pos1[0], 2) + pow(pos2[1] - pos1[1], 2) + pow(pos2[2] - pos1[2], 2));
  }
```

Once a full loop is made and identified in the maze using the is_back_at_start() function, the system reviews the data to pinpoint the area with the greatest light intensity above the LIGHT_INTENSITY_THRESHOLD. A specialized movement control system is used to navigate towards the target position, relying on GPS data and the distance_between_points() function to constantly monitor progress.

```
//function to check if robot has reached target position with maximum light intensity
bool is_at_max(const double *target_position, const PositionLightIntensity *best_position) {
  double dist = distance_between_points(target_position, (double[]){best_position->x,
  best_position->y, best_position->z});
  return dist < 0.01;  //return true if distance to best position is less than 0.01 meters
}
```

The is_at_max() function checks if the robot has reached the target position accurately within a threshold of 0.01 meters, at which time the motors are halted and the mission is deemed finished.

3.4. Movement Control & Position Tracking

```
double initial_position[3] = {0.0, 0.0, 0.0};  //initial position of robot
```

The GPS device is used by the robot's movement control system to constantly monitor its location in three dimensions. The starting position is saved at the beginning and serves as a point of reference to know when the robot has finished exploring the entire maze.

```
//function to check if robot has reached target position with maximum light intensity
bool is_at_max(const double *target_position, const PositionLightIntensity *best_position) {
  double dist = distance_between_points(target_position, (double[]){best_position->x,
  best_position->y, best_position->z});
  return dist < 0.01;  //return true if distance to best position is less than 0.01 meters
}
```

The position tracking system partners with the smooth_speed() function to guarantee accurate navigation without losing smooth movement. This system is especially important in the phase of navigating towards the target position, where the robot must effectively move through the maze to reach the designated point with the highest light intensity.

3.5. Data Management & Processing

Structured arrays and meticulous threshold management are key components of the implementation, ensuring robust data management. The recorded_positions array holds position coordinates and light intensity values, capped at 5000 entries to avoid overflow.

This information is constantly analyzed while navigating through the maze and is essential for reaching the spot with the highest level of light intensity. The system ensures reliable operation by checking boundaries and detecting duplicates to maintain data integrity during maze exploration and target navigation.

4. Conclusion

The successful demonstration of this self-navigating maze system has showcased important advancements in robot programming, as well as areas for possible improvement. The project successfully integrated various robotic control components, including accurate sensor integration, smart wall-following algorithms, seamless motion control, and target position navigation. Understanding how sensor fusion works in conjunction with proximity sensors, GPS, and light sensors has been crucial in achieving complex navigation tasks.

The algorithm for tracking the right-hand wall, along with the control system for smooth movement, has been successful in maneuvering through the maze and keeping motion stable. This implementation has offered valuable findings on the significance of gradual speed changes and accurate threshold adjustment for dependable robot performance. The importance of spatial awareness in autonomous navigation systems has been shown through the dead-end detection system and position tracking mechanisms, and the integration of multiple objectives into a single control system is exemplified by light intensity tracking and target position navigation.

5. References

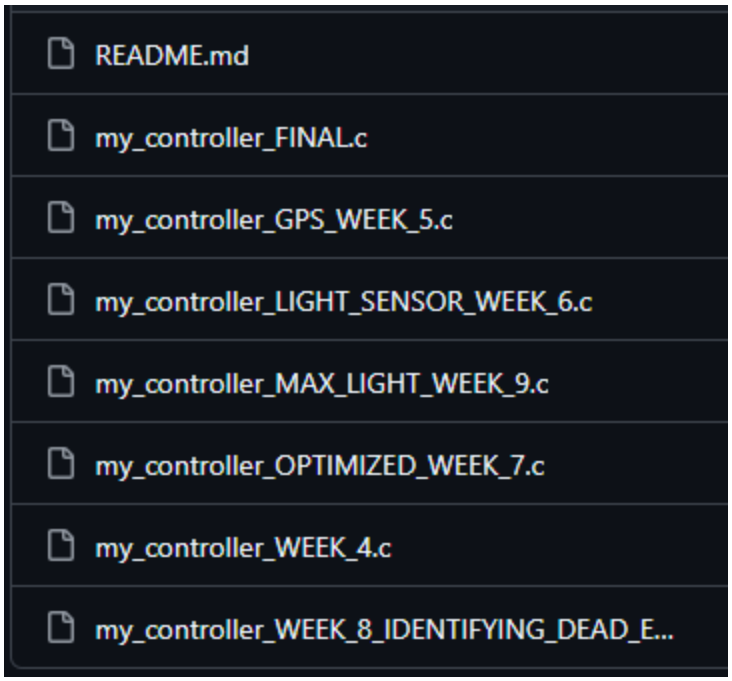
[1] Kajal Gada, "Wall follower webots C // Webots Tutorial project wall following e-puck in C // Kajal Gada", Youtube, 01/07/2021. [Online]. Available: https://www.youtube.com/watch?v=chamaHDJzCk&t=1013s.[Accessed: 11/11/2024]

6. Appendix A

The link attached here is the CSCI291 Project Repository. It has all the weekly progress made throughout the semester from week 4 onwards.

https://github.com/r-zk/CSCI291-Project

Below is a screenshot of the commits made:


README.md
my_controller_FINAL.c
my_controller_GPS_WEEK_5.c
my_controller_LIGHT_SENSOR_WEEK_6.c
my_controller_MAX_LIGHT_WEEK_9.c
my_controller_OPTIMIZED_WEEK_7.c
my_controller_WEEK_4.c
my_controller_WEEK_8_IDENTIFYING_DEAD_E...